EX 10: IMAGE GENERATION USING GENERATIVE ADVERSARIAL NETWORK (GAN)

AIM

To generate synthetic images using a pretrained StyleGAN2-ADA model and evaluate the quality of generated images using the Inception Score (IS) and Fréchet Inception Distance (FID).

PROCEDURE

1. **Import the required Python libraries and configure the execution device.**
2. **Clone the StyleGAN2-ADA repository and download the pretrained FFHQ generator.**
3. **Load the pretrained generator and generate synthetic images from random latent vectors.**
4. **Save the generated images in a designated directory.**
5. **Calculate the Inception Score to evaluate the quality and diversity of generated images.**
6. **Load real images from the CelebA dataset and save them in a separate directory.**
7. **Calculate the FID between real and generated images.**
8. **Compare the generated images visually and analyze the evaluation metrics.**

CODE

```
import os
import sys
import subprocess
import urllib.request

import torch
from torchvision import datasets, transforms
from torchvision.utils import save_image
from torch.utils.data import DataLoader
from torchmetrics.image.inception import InceptionScore

# Configure device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

REPO = "stylegan2-ada-pytorch"
REPO_URL = "https://github.com/NVlabs/stylegan2-ada-pytorch.git"

MODEL_URL = (
    "https://nvlabs-fi-cdn.nvidia.com/"
```

```
    "stylegan2-ada-pytorch/pretrained/ffhq.pkl"
)
MODEL_PATH = "ffhq.pkl"
NUM_IMAGES = 10


# Clone StyleGAN2-ADA repository
def clone_repo():
    if not os.path.exists(REPO):
        subprocess.run(
            ["git", "clone", REPO_URL],
            check=True
        )

    sys.path.insert(0, os.path.abspath(REPO))


# Download pretrained generator
def download_model():
    if not os.path.exists(MODEL_PATH):
        print("Downloading pretrained generator...")
        urllib.request.urlretrieve(MODEL_URL, MODEL_PATH)


# Generate synthetic images
def generate_images():
    import legacy

    with open(MODEL_PATH, "rb") as f:
        G = legacy.load_network_pkl(f)["G_ema"].to(device)

    os.makedirs("generated_images", exist_ok=True)
    images = []
```

```
    for i in range(NUM_IMAGES):
        z = torch.randn([1, G.z_dim], device=device)
        label = torch.zeros([1, G.c_dim], device=device)

        with torch.no_grad():
            img = G(
                z,
                label,
                truncation_psi=0.5,
                noise_mode="const"
            )

        img = (img + 1) / 2
        img = img.clamp(0, 1)

        save_image(img, f"generated_images/fake_{i}.png")
        images.append(img.cpu())

    return torch.cat(images, dim=0)


# Calculate Inception Score
def calculate_inception_score(images):
    metric = InceptionScore(normalize=True)

    # Convert images to uint8 and resize for evaluation
    images = torch.nn.functional.interpolate(
        images,
        size=(299, 299),
        mode="bilinear",
        align_corners=False
    )
```

```
    images = (images * 255).clamp(0, 255).to(torch.uint8)

    metric.update(images)
    mean, std = metric.compute()

    print(f"Inception Score: {mean.item():.2f} ± {std.item():.2f}")



# Prepare real images from CelebA
def prepare_real_images():
    transform = transforms.Compose([
        transforms.Resize((512, 512)),
        transforms.ToTensor()
    ])

    dataset = datasets.CelebA(
        root="./data",
        split="train",
        transform=transform,
        download=True
    )

    loader = DataLoader(dataset, batch_size=1, shuffle=True)
    os.makedirs("real_images", exist_ok=True)

    for i, (img, _) in enumerate(loader):
        if i >= NUM_IMAGES:
            break

        save_image(img, f"real_images/real_{i}.png")
```

```
# Calculate FID
def calculate_fid():
    subprocess.run(
        [
            sys.executable,
            "-m",
            "pytorch_fid",
            "real_images",
            "generated_images",
            "--device",
            str(device)
        ],
        check=True
    )

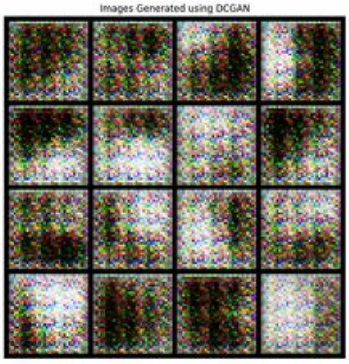

# Execute complete workflow
def main():
    clone_repo()
    download_model()

    images = generate_images()
    calculate_inception_score(images)

    prepare_real_images()
    calculate_fid()


if __name__ == "__main__":
    main()
```

OUTPUT





RESULT

Thus, synthetic images were generated using a pretrained StyleGAN2-ADA model. The quality of the generated images was evaluated using the Inception Score and Fréchet Inception Distance, demonstrating quantitative and visual approaches to evaluating generative models.